OpenFleX-Wheeled-Humanoid Robot User Manual (Basic)

English | 中文

> This document describes hardware control, VR teleoperation, and subsystem operation for the OpenFlex whole-body robot system. The full robot consists of a **four-wheel independent-steering omnidirectional chassis + lift-slide module + dual 7-DOF arms + 2-DOF head**, managed through a unified ros2_control architecture.
>
> Sensors, mapping, localization, navigation, VLA data collection/training/inference, and algorithm details are covered in the advanced manual.

Table of Contents

Chapter 1 Build Workspace

1.1 Build and Installation

If you are using the officially configured host, this chapter can be skipped. You can go directly to Chapter 2 Quick Start.

First create the workspace layout:

```
cd ~
mkdir openflex_all
cd openflex_all
mkdir openflex_maps openflex_models openflex_ws
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openflex_drivers.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openflex_vr_apk.git
cd openflex_ws
mkdir src
cd src
mkdir openflex_armx openflex_head openflex_integrated openflex_lift_slide
openflex_chassis
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-Humanoid/OpenFlex.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-Humanoid/openflex_EXO.git
cd openflex_armx
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_description.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_ros2.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_teleop_vr.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_tools.git
cd ..
cd openflex_chassis
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/base_model_interface_layer.git
```

```
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/hardware_sensor_layer.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/mapping_localization_layer.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/motion_control_layer.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/navigation_layer.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/system_bringup_layer.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/tools_common_layer.git
cd ..
cd openflex_head
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_head_bringup.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_head_description.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_head_hardware.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_head_teleop_vr_pico.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_head_tools.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_head_visio_h264.git
cd ..
cd openflex_integrated
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_integrated_bringup.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openarmx_integrated_description.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-Humanoid/openflex_gui.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openflex_manager.git
cd ..
cd openflex_lift_slide
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/lift_slide_bringup.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/lift_slide_description.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/lift_slide_driver.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/lift_slide_msgs.git
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/lift_slide_panel.git
cd ..
git clone -b v1.0_basic https://github.com/OpenFleX-Wheeled-
Humanoid/openflex_vr_bridge.git
python3 -m pip install openarmx_arm_driver -i
https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
```

Then add execute permission to the install script:

```
cd ~/openflex_all/openflex_ws/src/OpenFlex
chmod +x ./install_openflex_drivers_and_build.sh
```

Run the installation script. The script installs local dependency packages from `~/openflex_all/openflex_drivers`, builds ROS packages in dependency groups, and creates the desktop console shortcut.

```
cd ~/openflex_all/openflex_ws/src/OpenFlex
./install_openflex_drivers_and_build.sh
```

Chapter 2 Quick Start

2.1 Start the OpenFlex Console through the Desktop GUI

Right-click `OpenFlex 控制台` on the desktop and choose to allow launching. Then double-click `OpenFlex 控制台`.

(Insert image here later.)

You can also start it from a terminal:

```
cd ~/openflex_all/openflex_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run openflex_gui openflex_gui
```

2.2 Start Whole-Body Hardware from the Terminal

Before startup, confirm that all 6 CAN buses are powered and enabled. The default mapping is: right arm `can0`, left arm `can1`, head `can2`, lift-slide `can3`, chassis drive `can4`, and chassis steering `can5`.

Enable CAN interfaces: (You can also click `Enable All CAN` in the `OpenFlex 控制台`.)

```
python3
~/openflex_all/openflex_ws/src/openflex_integrated/openflex_manager/scripts/en_all_ca
n.py
```

Disable CAN interfaces: (You can also click `Disable All CAN` in the `OpenFlex 控制台`.)

```
python3
~/openflex_all/openflex_ws/src/openflex_integrated/openflex_manager/scripts/dis_all_c
an.py
```

Check motor status: (You can also click `Check Motor Status` in the `OpenFlex 控制台`.)

```
python3
~/openflex_all/openflex_ws/src/openflex_integrated/openflex_manager/scripts/check_mot
or_status.py
```

Start whole-body control:

```

```

```
cd ~/openflex_all/openflex_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=false \
  chassis_steering_can:=can5 \
  chassis_driving_can:=can4 \
  left_arm_can:=can1 \
  right_arm_can:=can0 \
  lift_can:=can3 \
  lift_node_id:=16 \
  head_can:=can2 \
  use_rviz:=true
```

2.3 Start VR Teleoperation

🧩 Install the Bridge App on Pico Connect the Device

1. Enable Developer Mode and USB debugging on your Pico device.
   Enable Developer Mode: `Settings > About Device > Tap software version repeatedly`
   Enable USB Debugging: `Settings > Developer Options > USB Debugging`
2. Connect Pico to your PC using a USB Type-C data cable.

Install the Pico Bridge APK

```
# Install ADB
sudo apt install adb

# Go to the APK directory
cd ~/openflex_all/openflex_vr_apk/apk/pico

# Install the bridge app
adb install OpenFlex.apk
```

Start VR teleoperation: (VR chassis velocity control is disabled by default.)

```
cd ~/openflex_all/openflex_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py
vr_chassis:=false
```

If VR chassis velocity control is required:

```
ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py
vr_chassis:=true
```

2.4 Quick Checks

```
ros2 control list_controllers
ros2 topic hz /joint_states
ros2 topic hz /pico_left_controller/pose
ros2 topic hz /pico_right_controller/pose
ip link show type can
```

Chapter 3 Whole-Body Management Tools

3.1 openflex_gui Whole-Body Management GUI

`openflex_gui` is a PyQt5-based OpenFlex whole-body management interface. It centralizes CAN bus management, motor status checks, whole-body ros2_control startup/shutdown, VR teleoperation startup/shutdown, and battery display. It is not a replacement for the lower-level launch files. It is a desktop panel that wraps common launch commands, CAN helper scripts, and status checks.

Code location: `src/openflex_integrated/openflex_gui/openflex_gui/main_window.py`

3.1.1 Installation

The installation script creates a desktop shortcut:

```
cd ~/openflex_all/openflex_ws/src/OpenFlex
./install_openflex_drivers_and_build.sh
```

Manual desktop shortcut creation:

```
cat > ~/桌面/OpenFlex控制台.desktop << EOF
[Desktop Entry]
Version=1.0
Type=Application
Name=OpenFlex 控制台
Exec=bash -c "source /opt/ros/humble/setup.bash && source
\$HOME/openflex_all/openflex_ws/install/setup.bash && ros2 run openflex_gui
openflex_gui"
Icon=$HOME/openflex_all/openflex_ws/src/openflex_integrated/openflex_gui/openflex_gui
/openflex_vr.svg
Terminal=false
Categories=Robotics;
EOF
chmod +x ~/桌面/OpenFlex控制台.desktop
```

3.1.2 Startup

Start from terminal:

```
cd ~/openflex_all/openflex_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run openflex_gui openflex_gui
```

Start through the desktop shortcut: the shortcut name is `OpenFlex 控制台`.

3.1.3 Function Areas

The GUI currently contains 6 function areas:

Area	Buttons / Options	Actual behavior
CAN bus	`Enable All CAN`, `Disable All CAN`	Calls helper scripts in the package to configure `can0`~`can5` at 1 Mbps
Motor status	`Check All Motor Status`	Checks responses from lift-slide, chassis, dual arms, and head motors through CAN tools in `openflex_manager/scripts`
Whole-body control	`Start Whole-Body Control`, `Stop`	Starts or stops `openarmx_integrated_bringup integrated_robot_bringup.launch.py`
VR teleoperation	`Start VR Teleoperation`, `Stop`, `VR controls chassis velocity`	Starts `openarmx_integrated_bringup integrated_vr_teleop.launch.py`. If chassis velocity is checked, it passes `vr_chassis:=true`; otherwise it passes `vr_chassis:=false`
Keyboard chassis control	`Start Keyboard Chassis Control`, `Stop`	Starts `swerve_bringup swerve_teleop.py` for keyboard chassis teleoperation
Battery display	`Show Battery`, `Stop`	Configures battery serial permission and starts `openarmx_battery_monitor auto_pack_overlay.launch.py`

3.1.4 Whole-Body Control Command

When `Start Whole-Body Control` is clicked, the GUI executes this core command:

```
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=false \
  chassis_steering_can:=can5 \
  chassis_driving_can:=can4 \
  left_arm_can:=can1 \
  right_arm_can:=can0 \
  lift_can:=can3 \
  lift_node_id:=16 \
  head_can:=can2 \
  use_rviz:=true
```

When `Stop` is clicked, the GUI first tries to switch the chassis controller and chassis hardware to inactive:

```
ros2 control set_controller_state swerve_drive_controller inactive
ros2 control set_hardware_component_state swerve_drive_system inactive
```

Then it sends SIGINT to the whole-body bringup process. If the process does not exit in time, it is forcibly terminated.

3.1.5 VR Startup Logic

The VR area has two startup paths:

VR controls chassis velocity	Startup command	Description
Unchecked	`ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py vr_chassis:=false`	Starts the whole-body VR teleoperation wrapper, suitable for regular VR operation
Checked	`ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py vr_chassis:=true`	Uses the VR teleop launch in integrated bringup and enables VR chassis velocity control

After whole-body control is started, the GUI checks whether key controllers in `/controller_manager` are active. After they are ready, VR can be started. The current checks include:

- `joint_state_broadcaster`
- `swerve_drive_controller`
- `velocity_controller`
- `left_forward_position_controller`
- `right_forward_position_controller`
- `head_forward_position_controller`

3.1.6 Status Lights and Logs

Each function area has a status light on the left:

State	Meaning
Gray	Idle or not started
Green	Running normally or operation succeeded
Red	Operation failed, process exited abnormally, or status check failed

The log window at the bottom outputs helper script messages, launch process messages, and status check results. If an exception occurs, first check the red error lines in the log window.

3.2 openflex_manager Whole-Body Motor Management Tool

`openflex_manager` is the OpenFlex whole-body management tool package. It provides CAN interface enable/disable helpers, motor status checks, and related whole-body maintenance functions.

Code location: `src/openflex_integrated/openflex_manager/`

3.2.1 Main Features

Feature	Description
Motor status check	Checks whether lift-slide, chassis, dual-arm, and head motors are online
CAN interface enable	One-click `ip link set canX up`
CAN interface disable	One-click `ip link set canX down`
Configuration management	Reads the OpenFlex default CAN mapping and local configuration

3.2.2 Default Whole-Body CAN Mapping

The following mapping is the current default wiring and startup parameter layout for the OpenFlex whole-body robot. It is consistent with `openflex_gui`, `integrated_robot_bringup.launch.py`, and the whole-body motor management scripts. Note that the arms are not `can0=left arm` and `can1=right arm`; the current default is right arm on `can0` and left arm on `can1`.

CAN interface	Baudrate	Purpose
can0	1 Mbps	Right arm
can1	1 Mbps	Left arm
can2	1 Mbps	Head
can3	1 Mbps	Lift-slide
can4	1 Mbps	Chassis drive UM hub motors
can5	1 Mbps	Chassis steering RS06

3.3 Common System Maintenance Commands

3.3.1 CAN Interface Management

```
# Bring up one CAN interface at 1 Mbps
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

# Bring up all CAN interfaces
for i in 0 1 2 3 4 5; do
  sudo ip link set can${i} type can bitrate 1000000 && sudo ip link set can${i} up
done

# Show CAN status
ip link show type can

# Show CAN traffic
candump can0
```

3.3.2 ROS 2 System Checks

```
# Check all controller states
ros2 control list_controllers

# Key topic rates
ros2 topic hz /joint_states
ros2 topic hz /odom
ros2 topic hz /cmd_vel

# TF check
ros2 run tf2_ros tf2_echo base_link lift_carriage_link

# Lift-slide services
ros2 service call /lift_slide_driver/enable std_srvs/srv/Trigger
ros2 service call /lift_slide_driver/start_homing std_srvs/srv/Trigger

# Dynamically modify dual-arm KP
```

```
ros2 param set /openarmx_left_hardware_params kp_joint1 30.0
ros2 param set /openarmx_right_hardware_params kp_joint1 30.0
```

3.3.3 VR System Checks

```
# Check UDP port listener
sudo netstat -tunlp | grep 5100

# Check VR topics
ros2 topic hz /pico_left_controller/pose
ros2 topic hz /pico_right_controller/pose
ros2 topic hz /pico_head/pose
```

3.3.4 Whole-Body Troubleshooting Order

```
1. Source the environment
   source /opt/ros/humble/setup.bash
   source install/setup.bash

2. Check CAN interfaces
   ip link show type can

3. Check motor responses
   candump can0 | head

4. Check controller states
   ros2 control list_controllers

5. Check key topics
   ros2 topic hz /joint_states

6. Check VR input
   ros2 topic hz /pico_left_controller/pose
```

Chapter 4 Whole-Body Motion Control System

4.1 Whole-Body Control Quick Start

4.1.1 Start Whole-Body Hardware

Before startup, confirm that all 6 CAN buses are enabled and powered. Default mapping: right arm `can0`, left arm `can1`, head `can2`, lift-slide `can3`, chassis drive `can4`, chassis steering `can5`.

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=false \
  chassis_steering_can:=can5 \
  chassis_driving_can:=can4 \
  left_arm_can:=can1 \
  right_arm_can:=can0 \
```

```
  lift_can:=can3 \
  head_can:=can2
```

If real hardware is not available, mock mode can be used:

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=true
```

4.1.2 Checks after Startup

```
ros2 control list_controllers
ros2 topic hz /joint_states
ros2 topic list | grep -E "cmd_vel|joint_states|controller|lift|head"
ros2 service list | grep lift_slide_driver
```

Observation	Meaning
`joint_state_broadcaster` is active	Whole-body joint state broadcasting is normal
`swerve_drive_controller` is active	Chassis controller is loaded
Left/right arm position controllers are active	Dual-arm controllers are loaded
`head_forward_position_controller` is active	Head controller is loaded

4.2 System Layers and Code Organization

```
src/
├── openflex_chassis/              ← Chassis subsystem (Chapter 6)
│   ├── base_model_interface_layer/swerve_description/
│   ├── hardware_sensor_layer/swerve_hardware/
│   ├── motion_control_layer/swerve_controller/
│   ├── system_bringup_layer/swerve_bringup/
│   ├── navigation_layer/swerve_navigation/
│   ├── navigation_layer/nmpc_controller/
│   └── mapping_localization_layer/
├── openflex_lift_slide/           ← Lift-slide subsystem (Chapter 7)
│   ├── lift_slide_description/
│   ├── lift_slide_driver/
│   ├── lift_slide_msgs/
│   └── lift_slide_panel/
├── openflex_armx/                 ← Dual-arm subsystem (Chapter 8)
│   ├── openarmx_description/
│   ├── openarmx_ros2/openarmx_hardware/
│   ├── openarmx_teleop_vr/
│   └── openarmx_tools/
├── openflex_head/                 ← Head subsystem (Chapter 9)
│   ├── openarmx_head_description/
│   ├── openarmx_head_hardware/
│   ├── openarmx_head_teleop_vr_pico/
│   └── openarmx_head_visio_h264/
```

```
├── openflex_integrated/           ← Integration layer
│   ├── openarmx_integrated_bringup/
│   ├── openarmx_integrated_description/
│   └── openflex_gui/
└── openflex_vr_bridge/            ← VR controller UDP bridge
```

4.3 Integrated URDF and TF Tree

Integrated URDF:
`src/openflex_integrated/openarmx_integrated_description/urdf/openarmx_integrated_robot.urdf.xacro`

4.3.1 Whole-Body Mechanical Chain

```
odom
└── base_link  (chassis center)
    ├── fl/fr/bl/br_steering_link → *_wheel_link    (four wheels)
    ├── mid360_link → livox_frame                   (LiDAR)
    └── lift_base_link  (fixed connection, z=+0.07m)
        └── lift_joint  (prismatic, -0.650 ~ 0.300m)
            └── lift_carriage_link  (lift-slide platform)
                ├── left_link0_base → ... → left_link7 (left arm 7 DOF + gripper)
                ├── right_link0_base → ... → right_link7 (right arm 7 DOF + gripper)
                └── head_base_link → head_pitch_link → head_yaw_link (2-DOF head)
```

> **Design note:** Both arms and the head are mounted under `lift_carriage_link`, so lift-slide motion moves the arms and head together.

4.4 Integrated controller_manager and Controller List

Config file:
`src/openflex_integrated/openarmx_integrated_bringup/config/integrated_controllers.yaml`

All subsystems share one `controller_manager`, running at 100 Hz.

Controller name	Type	Joints	Description
`joint_state_broadcaster`	JointStateBroadcaster	All joints	Broadcasts to `/joint_states`
`swerve_drive_controller`	SwerveDriveController	8 chassis joints	Chassis motion control and odometry
`lift_position_controller`	ParamForwardingPositionController	`lift_joint`	Lift-slide position control
`lift_manual_position_controller`	LiftSlideManualPositionController	`lift_joint`	Lift-slide manual jog and step position control

Controller name	Type	Joints	Description
velocity_controller	ParamForwardingVelocityController	lift_joint	Legacy velocity controller, inactive by default
left_forward_position_controller	JointGroupPositionController	8 left-arm joints	Left-arm position control
right_forward_position_controller	JointGroupPositionController	8 right-arm joints	Right-arm position control
head_forward_position_controller	ForwardCommandController	2 head joints	Head position control

Command Topics

Topic	Message type	Controller
/cmd_vel	Twist	swerve_drive_controller
/lift_position_controller/commands	Float64MultiArray	lift_position_controller
/lift_manual_position_controller/jog_command	Float64	lift_manual_position_controller
/lift_manual_position_controller/step_command	Float64MultiArray	lift_manual_position_controller
/left_forward_position_controller/commands	Float64MultiArray	Left arm
/right_forward_position_controller/commands	Float64MultiArray	Right arm
/head_forward_position_controller/commands	Float64MultiArray	Head

4.5 CAN Bus Planning

CAN interface	Subsystem	Protocol	Baudrate	Motors / devices
can0	Right arm	Robstride custom	1 Mbps	RS04×2 + RS03×2 + RS00×3 + RS00 gripper
can1	Left arm	Robstride custom	1 Mbps	Same as right arm
can2	Head	Robstride custom	1 Mbps	RS00×2 (yaw + pitch)
can3	Lift-slide	CANopen (CiA 402)	1 Mbps	Node ID 16
can4	Chassis drive	CANopen standard frame	1 Mbps	UM hub motors ×4 (ID 1/2/3/4)
can5	Chassis steering	RS06 custom extended frame	1 Mbps	RS06×4 (ID 5/6/7/8)

Power-on CAN initialization:

```
for i in 0 1 2 3 4 5; do
  sudo ip link set can${i} type can bitrate 1000000 && sudo ip link set can${i} up
done
```

Chapter 5 Whole-Body VR Teleoperation System

5.1 Whole-Body VR Teleoperation Quick Start

5.1.1 Prerequisites

Start the whole-body hardware control chain first:

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=false \
  chassis_steering_can:=can5 \
  chassis_driving_can:=can4 \
  left_arm_can:=can1 \
  right_arm_can:=can0 \
  lift_can:=can3 \
  head_can:=can2
```

5.1.2 Start VR Teleoperation

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py
```

Feature	Parameter	Default	Description
Dual-arm IK	-	Always enabled	Cannot be disabled
Joystick chassis	`enable_joystick_control`	`true`	Left joystick translation, right joystick steering
Button lift	`enable_button_lift`	`true`	Left-hand X/Y control
Head tracking	`enable_head_teleop`	`true`	VR headset controls head
Waist control	`enable_waist_control`	`false`	Disabled by default

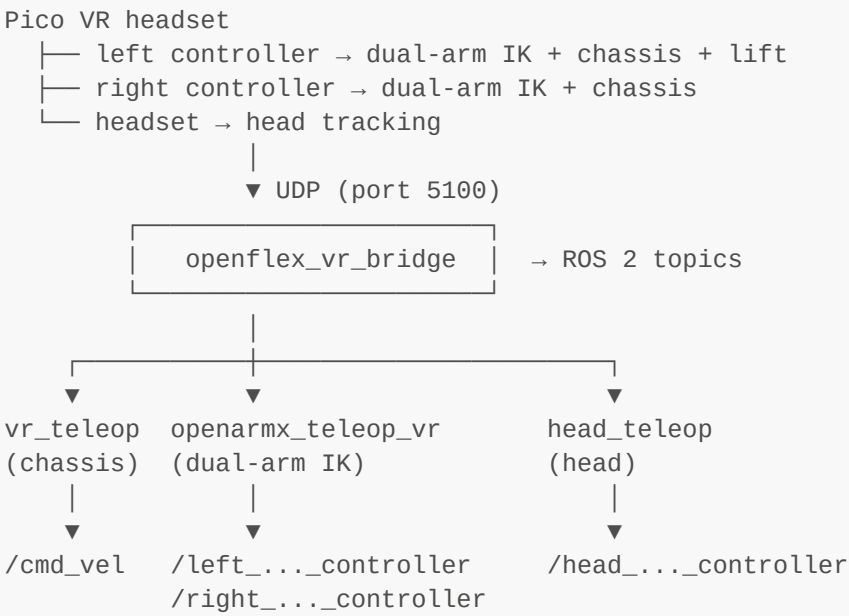
5.1.3 Operation Checks

```
ros2 topic hz /pico_left_controller/pose
ros2 topic hz /cmd_vel
ros2 topic hz /left_forward_position_controller/commands
ros2 topic hz /head_forward_position_controller/commands
```

Operation	Effect
Move left/right controllers	Controls dual-arm end-effector poses
Trigger/grip	Controls gripper opening/closing

Operation	Effect
Left joystick	Chassis forward/backward and lateral translation
Right joystick X	Chassis rotation
Left-hand X/Y	Lift-slide down/up
Headset rotation	Head yaw/pitch follow

5.2 Overall Architecture



VR launch file:
`src/openflex_integrated/openarmx_integrated_bringup/launch/integrated_vr_teleop.launch.py`

5.3 openflex_vr_bridge UDP Protocol

Code location: `src/openflex_vr_bridge/src/pose_bridge_node.cpp`

Parameter	Default	Description
Listen address	0.0.0.0	Receives on all network interfaces
Listen port	5100	UDP port

UDP Datagram Format

Full controller packet:

```
HAND LEFT x y z qx qy qz qw trigger grip button_a button_b button_x button_y
joystick_click joystick_x joystick_y rate timestamp_ns
```

Incremental update packets:

```
BTN LEFT/RIGHT A/B/X/Y/J pressed timestamp_ns
JOY LEFT/RIGHT joystick_x joystick_y timestamp_ns
TRIG LEFT/RIGHT trigger_value timestamp_ns
```

Body trackers:

```
WAIST x y z qx qy qz qw timestamp_ns
HEAD x y z qx qy qz qw timestamp_ns
```

Published ROS 2 Topics

Topic	Type	Description
`/pico_left_controller/pose`	PoseStamped	Left-hand 6-DOF pose
`/pico_left_controller/trigger`	Float32	Trigger value (0~1)
`/pico_left_controller/grip`	Float32	Grip value (0~1)
`/pico_left_controller/joystick_x`	Float32	Joystick X axis
`/pico_left_controller/joystick_y`	Float32	Joystick Y axis
`/pico_left_controller/rate`	Float32	Speed mode
`/pico_head/pose`	PoseStamped	Head pose
`/pico_tracker/waist/pose`	PoseStamped	Waist pose

5.4 Dual-Arm VR IK Node

Code location:
`src/openflex_armx/openarmx_teleop_vr/openarmx_teleop_vr/openarmx_teleop_vr_node.py`

Parameter	Default	Description
`control_rate`	100.0 Hz	IK solve rate
`grip_threshold`	0.5	Gripper close threshold
`resync_threshold_deg`	5.0 deg	Resynchronization threshold
`ik_iterations`	3	Number of IK iterations
`sync_joint_states_each_cycle`	true	Synchronize joint states every cycle
`max_step_deg_joint1_2`	8.0 deg/cycle	Maximum step for joints 1-2
`max_step_deg_joint3_4`	5.0 deg/cycle	Maximum step for joints 3-4
`max_step_deg_joint5_7`	5.0 deg/cycle	Maximum step for joints 5-7

Output topics:

Topic	Data
`/left_forward_position_controller/commands`	`[j1..j7, finger]`

Topic	Data
`/right_forward_position_controller/commands`	`[j1..j7, finger]`

5.5 Head VR Node

Code location:
`src/openflex_head/openarmx_head_teleop_vr_pico/openarmx_head_teleop_vr_pico/head_teleop_node.py`

Parameter	Default	Description
`slow_max_step_deg`	2.0 deg/cycle	Slow step
`fast_max_step_deg`	5.0 deg/cycle	Fast step
`yaw_scale` / `pitch_scale`	1.0	Sensitivity
`enable_soft_limits`	true	Enables soft limits
`startup_home_enabled`	true	Enables startup auto-homing

Control logic: right-hand Button B toggles relative control on/off. When enabled, the node records an anchor quaternion, computes relative rotation, extracts yaw/pitch, applies step limits and soft limits, and then publishes commands.

5.6 Joystick Chassis Control Node

Code location:
`src/openflex_chassis/system_bringup_layer/swerve_bringup/scripts/vr_teleop_node.py`

Input	Mapping	Range
Left joystick Y	`linear.x` forward/backward	±max_linear_speed
Left joystick X	`linear.y` left/right translation	±max_linear_speed
Right joystick X	`angular.z` rotation	±max_angular_speed

Button	Function
Right-hand A	Toggle chassis enable
Right-hand B	Emergency stop and disable

Safety logic: if no VR data is received for more than 0.5 s, the node automatically sends zero velocity.

5.7 Lift Button Control Node

Code location:
`src/openflex_chassis/system_bringup_layer/swerve_bringup/scripts/vr_lift_control_node.py`

Button	Function
Left-hand X	Down jog
Left-hand Y	Up jog

Output: `/lift_manual_position_controller/jog_command` (Float64). When pressed, the node publishes signed jog speed. When released, conflicting, timed out, or emergency-stopped, it publishes `0.0` to stop. The lower layer still uses `LiftSlideManualPositionController` to generate continuous position targets.

5.8 Waist Closed-Loop Position Control Node

Code location:
`src/openflex_chassis/system_bringup_layer/swerve_bringup/scripts/waist_chassis_control_node.py`

Activation condition: active when the left or right trigger is >= 0.5.

Control principle, using closed-loop incremental target pose:

1. Record the anchor when activated
2. Compute waist delta relative to the anchor: Δx, Δy, Δyaw, Δlift
3. Closed-loop output to `/cmd_vel` and `/lift_position_controller/commands`

5.9 VR Emergency Stop and Safety Strategy

Mechanism	Description
VR signal-loss protection	Chassis sends zero velocity if no data is received for more than 0.5 s
Emergency stop button	Right-hand B emergency-stops and disables the chassis
Step limiting	Dual arms and head both have `max_step_deg`
Soft limits	Head buffers and slows down near limits
IK joint-limit clipping	Solutions outside URDF joint limits are clipped

---

Chapter 6 Chassis Subsystem

6.1 Chassis Quick Start

```
# Start chassis motion control
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch swerve_bringup swerve_drive.launch.py

# In another terminal, start keyboard teleop
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 run swerve_bringup swerve_teleop.py
```

Parameter	Default	Description
`use_rviz`	`true`	Whether to start RViz
`steering_can_interface`	`can5`	Steering CAN
`driving_can_interface`	`can4`	Drive CAN
`max_wheel_speed`	`2.0`	Wheel speed limit, m/s

Parameter	Default	Description
`wheel_accel_limit`	`1.2`	Wheel speed rate limit, m/s²

Keyboard operation: `i` forward, `,` backward, `j` turn left, `l` turn right, `k` stop, `a` translate left, `d` translate right.

6.2 Mechanical Structure and Dimensions

The chassis uses four-wheel independent steering and independent driving (Swerve Drive). Each wheel module consists of one RS06 steering motor and one UM hub servo motor, for a total of 8 motors.

Parameter	Value
Chassis length × width × height	0.50 × 0.40 × 0.14 m
Track width	0.52 m
Wheelbase	0.42 m
Wheel radius	0.14 m outer radius / 0.075 m effective radius
Chassis mass	20.0 kg

Wheel module positions relative to `base_link`: FL(+0.21,+0.26), FR(+0.21,-0.26), BL(-0.21,+0.26), BR(-0.21,-0.26)

6.3 URDF Model Structure

File location:
`src/openflex_chassis/base_model_interface_layer/swerve_description/urdf/swerve.urdf.xacro`

```
base_footprint → base_link
    ├── fl_steering_joint(revolute,Z) → fl_steering_link →
fl_wheel_joint(continuous,Y) → fl_wheel_link
    ├── fr/bl/br same as above
    ├── mid360_link → livox_frame
    └── d435_link
```

Steering joint limit: ±1.5708 rad (±90°)

6.4 Hardware Interface and CAN Communication

Code location:
`src/openflex_chassis/hardware_sensor_layer/swerve_hardware/src/swerve_drive_hardware.cpp`

```
SwerveDriveHardware (SystemInterface)
    ├── CAN5 (extended frame) ── RS06 steering motors ×4 (ID: 5/6/7/8)
    └── CAN4 (standard frame) ── UM hub servos ×4 (ID: 1/2/3/4)
```

RS06 initialization sequence: MOTOR_STOP → RUN_MODE=5(CSP) → LIMIT_SPD=4.0 → LIMIT_CUR=8.0 → LOC_REF=zero → MOTOR_ENABLE

UM drive mode: Profile Velocity (PV), CiA 402 state machine. Controlword 0x0006→0x0007→0x000F enters Operation Enabled.

6.5 Kinematics Algorithm

Code location:
`src/openflex_chassis/motion_control_layer/swerve_controller/src/swerve_drive_kinematics.cpp`

Inverse Kinematics (`cmd_vel` to wheel module states)

```
module_vx_i = vx - ω × yi
module_vy_i = vy + ω × xi
target_angle_i = atan2(module_vy_i, module_vx_i)
target_wheel_speed_i = sqrt(module_vx_i² + module_vy_i²)
```

Key Optimizations

1. **Angle locking:** when `cmd_vel` is zero, reuse the previous command angle to prevent jitter
2. **Angle-difference normalization:** `atan2(sin(diff), cos(diff))` handles the ±π boundary
3. **Steering-error speed attenuation:** `cos(error)^exponent`; speed is set to zero when the error is too large
4. **Acceleration limiting:** per-cycle `wheel_accel_limit × dt`
5. **Speed desaturation:** if speed exceeds `max_wheel_speed`, all speeds are scaled down proportionally

---

6.6 Odometry Calculation

Forward kinematics uses least squares to compute `(vx, vy, ω)` from the 4 wheel module states, then integrates pose using second-order midpoint integration:

```
mid_heading = heading + omega*dt/2
x += (vx*cos(mid_heading) - vy*sin(mid_heading)) * dt
y += (vx*sin(mid_heading) + vy*cos(mid_heading)) * dt
```

Published topic: `/odom`, at 50 Hz.

6.7 Controller Parameters

Config location: `src/openflex_chassis/system_bringup_layer/swerve_bringup/config/`

Parameter	Navigation mode	Mapping mode	Description
`enable_odom_tf`	false	true	Whether to publish odom→base_link TF
`max_wheel_speed`	1.2	2.0	Wheel speed limit, m/s
`wheel_accel_limit`	1.0	1.2	Acceleration limit
`cmd_vel_timeout`	0.5	0.5	Stop on command timeout
`steering_align_threshold`	0.08	0.08	Steering alignment threshold, rad
`steering_stop_threshold`	0.90	0.90	Single-wheel drive stop threshold

Chapter 7 Lift-Slide Subsystem

7.1 Mechanics and Hardware

Parameter	Value	Description
Stroke	-0.650 ~ 0.300 m	Home-relative lift-slide soft-limit range
Mounting offset	z = +0.07 m	`lift_base_link` relative to `base_link`
Joint type	prismatic	Z-axis direction
Joint name	`lift_joint`	
Communication interface	CAN3	CANopen protocol
Node ID	16	
Position conversion	2,000,000 counts/m	

File locations:

- URDF: `src/openflex_lift_slide/lift_slide_description/urdf/lift_slide_module.urdf.xacro`
- Hardware interface: `src/openflex_lift_slide/lift_slide_driver/src/lift_slide_hardware_interface.cpp`

7.2 CANopen Protocol and Object Dictionary

Object index	Name	Description
0x6040	Controlword	Control word
0x6041	Statusword	Status word
0x6060	Mode of Operation	Operation mode
0x6064	Position Actual	Actual position
0x606C	Velocity Actual	Actual velocity
0x607A	Target Position	Target position
0x60FF	Target Velocity	Target velocity
0x60FD	Digital Inputs	Limit switches

CiA 402 power-on sequence: Shutdown(0x0006) → Switch On(0x0007) → Enable Operation(0x000F)

7.3 ros2_control Interfaces and Controllers

Plugin name: `lift_slide_driver/LiftSlideHardwareInterface`

State interfaces: position, velocity, cia402_state, statusword, is_enabled, is_fault, homing_state, homing_complete, upper/home/lower_limit_switch

Command interfaces: position, velocity

Controller	Type	Description
`lift_manual_position_controller`	LiftSlideManualPositionController	**Active by default**, used by the RViz panel and VR button jog

Controller	Type	Description
`lift_position_controller`	ParamForwardingPositionController	Position control, used by waist control and direct position commands
`velocity_controller`	ParamForwardingVelocityController	Legacy velocity controller, inactive by default
`lift_state_controller`	LiftSlideStateController	State publishing and speed-parameter compatibility topics

7.4 Automatic Homing Flow

The lift-slide has no absolute encoder. Each power cycle requires homing to establish a position reference.

Parameter	Value
`homing_method`	27
`homing_speed`	0.010 m/s
`homing_timeout`	60.0 s
`home_switch_position_m`	0.650 m

When `auto_homing:=true` is used in `integrated_robot_bringup.launch.py`:

```
+9.0s   call /lift_slide_driver/enable → enable
+12.0s  call /lift_slide_driver/start_homing → homing
          └── detect home_switch → stop → set current position
```

7.5 State, Limits, and Service Interfaces

Service name	Description
`/lift_slide_driver/enable`	Enable the drive
`/lift_slide_driver/start_homing`	Start homing
`/lift_slide_driver/return_home`	Return to home
`/lift_slide_driver/quick_stop`	Emergency stop

Topic	Description
`/lift_slide_driver/motor_status`	Motor status
`/lift_slide_driver/homing_state`	Homing progress
`/lift_slide_driver/limit_switch_state`	Limit switch state

Chapter 8 Dual-Arm Subsystem (OpenArmX)

8.1 Mechanical Structure and Motor Configuration

Each arm has 7 revolute joints plus 1 gripper, for 8 DOF. Both arms together have 16 DOF.

Joint	CAN ID	Motor model	Description
Joint 1	0x01	RS04	Shoulder yaw
Joint 2	0x02	RS04	Shoulder pitch
Joint 3	0x03	RS03	Upper-arm rotation
Joint 4	0x04	RS03	Elbow
Joint 5	0x05	RS00	Forearm rotation
Joint 6	0x06	RS00	Wrist yaw
Joint 7	0x07	RS00	Wrist pitch
Gripper	0x08	RS00	End gripper

CAN assignment: left arm `can1`, right arm `can0`

8.2 Joint Limits and Kinematics

Joint	Lower limit (rad)	Upper limit (rad)	Velocity (rad/s)
Joint 1	-1.25	3.0	10.47
Joint 2	-1.70	1.7	10.47
Joint 3	-1.57	1.57	10.47
Joint 4	0.0	1.8	10.47
Joint 5	-1.50	1.50	10.47
Joint 6	-0.75	0.75	10.47
Joint 7	-1.40	1.40	10.47

Config file: `src/openflex_armx/openarmx_description/config/arm/v10/joint_limits.yaml`

8.3 ros2_control Hardware Interface Implementation

Plugin name: `openarmx_hardware/OpenArmX_v10HW`
Code file: `src/openflex_armx/openarmx_ros2/openarmx_hardware/src/v10_simple_hardware.cpp`

Callback	Action
`on_init()`	Parses parameters, creates ROS 2 parameter node for KP/KD, initializes CAN
`on_activate()`	Enables all motors and reads current positions as initial command values
`on_deactivate()`	Disables all motors
`read()`	Reads position/velocity/torque, applies direction multipliers, maps gripper rad→m
`write()`	MIT: sends MotionControlParam; CSP: sends LOC_REF

8.4 MIT and CSP Control Modes

MIT mode (default): sends `{position, velocity=0, torque=0, kp, kd}` every cycle. Stiffness and damping can be adjusted online.

CSP mode: sends only target position `LOC_REF` every cycle. The drive closes the loop internally and produces smoother trajectories.

8.5 Arm Prefixes and Bimanual Configuration

Parameter	Left arm	Right arm
`arm_prefix`	`left_`	`right_`
`can_interface`	can1	can0

Joint name format: `openarmx_{prefix}joint{N}`, for example `openarmx_left_joint1`.

8.6 Gripper Mapping

The gripper joint in URDF is prismatic and is driven by a revolute motor:

Direction	Formula
Joint to motor	`motor_rad = (joint_m / 0.044) × 1.0472`
Motor to joint	`joint_m = 0.044 × (motor_rad / 1.0472)`

Joint travel: 0 ~ 0.044 m. Motor travel: 0 ~ 60°.

8.7 Dynamic KP/KD Parameters

In MIT mode, parameters can be adjusted at runtime through the parameter service:

Joint	Default KP	Default KD
Joint 1~4	50.0	2.5
Joint 5~7	10.0	0.5
Gripper	50.0	2.5

```
ros2 param set /openarmx_left_hardware_params kp_joint1 30.0
ros2 param set /openarmx_right_hardware_params kp_joint1 30.0
ros2 param set /openarmx_left_hardware_params kd_joint5 1.0
ros2 param set /openarmx_right_hardware_params kd_joint5 1.0
```

Scenario	KP	KD	Description
VR teleoperation	50/10	2.5/0.5	High-stiffness tracking
Compliant grasping	10/3	1.0/0.3	Low-stiffness adaptation
Force-control experiment	0	0.5~2.0	Pure damping

Chapter 9 Head Subsystem

9.1 Mechanics and Motors

2 DOF: pitch + yaw, mounted above `lift_carriage_link`.

Joint	CAN ID	Motor	Mounting position
openarmx_head_pitch_joint	0x02	RS00	Bottom motor
openarmx_head_yaw_joint	0x01	RS00	Top motor

Communication interface: CAN2. Default KP=100.0, KD=10.0.

9.2 URDF Link Structure

File: src/openflex_head/openarmx_head_description/urdf/head.urdf.xacro

```
head_base_link
  └── openarmx_head_pitch_joint (revolute, X axis, ±90°)
      └── head_pitch_link
          └── openarmx_head_yaw_joint (revolute, Z axis, ±90°)
              └── head_yaw_link (contains camera)
```

9.3 ros2_control Head Hardware Interface

Plugin name: openarmx_head_hardware/OpenArmX_HeadHW
Code file: src/openflex_head/openarmx_head_hardware/src/head_hardware.cpp

Parameter	Default	Description
can_interface	can2	Head CAN
control_mode	csp	mit / csp
home_on_activate	true	Automatically soft-home on activation
home_duration_sec	3.0	Soft-homing duration

Controller: head_forward_position_controller. Command topic: /head_forward_position_controller/commands. Data format: [yaw_rad, pitch_rad].

9.4 Startup Soft Homing

The head motors execute soft homing at startup to avoid sudden motion:

```
During on_activate():
1. Read current position (yaw_curr, pitch_curr)
2. Linearly interpolate to zero position within home_duration_sec (3.0s)
3. Switch to normal control mode after completion
```

9.5 Head Camera and Video Streaming (H.264)

Code location: src/openflex_head/openarmx_head_visio_h264/

Current VR video protocol: UDP + OAR3 fragmentation format, default port 5600, H.264 encoding.

Startup:

```
# Full startup: camera + VR streaming
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_head_vision_h264 d435i_vr.launch.py

# Camera only
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_head_vision_h264 d435i_source.launch.py

# VR streaming only
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_head_vision_h264 vr_forwarder_only.launch.py \
  image_topic:=/vision/color/image_raw udp_port:=5600
```

Preset mode	Frame rate	Bitrate	Use case
`balanced`	20 fps	4000 kbps	Default
`high_quality`	25 fps	6000 kbps	High quality
`low_latency`	25 fps	2500 kbps	Very low latency
`bandwidth_saving`	15 fps	1500 kbps	Weak network

End of document. Sensors, mapping, localization, navigation, VLA data collection/training/inference, and algorithm details are covered in the advanced manual.